ReadyOS SHIM Architecture: 0.1.8 Runtime Reality

Implementation-backed private report for the active launcher + shim + REU model. This canonical report is intentionally fuller than the one-screen summary docs: it keeps the resident byte map, the annotated shim routine walkthrough, the profile-driven catalog/release contract, and the ABI vs implementation nuances that are easy to lose in condensed summaries. Current checked-in artifacts are stamped `0.1.0G`, that suffix is a local build stamp, not a separate public release line.

Executive Snapshot

RELEASE	SHIM WINDOW	SHIM SIZE	APP WINDOW	SNAPSHOT SIZE
`0.1.0 / 0.1.0G local`	`$C000-$C9FF`	`512 bytes`	`$1000-$C3FF`	`$0600`

Important correction: in the current runtime, REU bank **0** is launcher-owned. Shared system metadata such as the allocation table, clipboard metadata, hotkey bindings, and shim globals live in C64 RAM at `$C600-$C9FF`, not inside REU bank **0**.

Canonical C64 RAM Map

`$1000-$C3FF`	Active app snapshot/load window	`$0600`
`$C000-$C7FF`	REU metadata RAM, clipboard tables, debug bytes, hotkey bindings	512 B
`$C800-$C9FF`	Resident shim jump table, globals, and swap/load routines	512 B
`$D000-$DFFF`	I/O and hardware registers	4 KB

Shared RAM highlights

Address	Purpose
`$C400-$C4FF`	REU allocation table
`$C700`	REU manager magic
`$C782 / $C718`	clipboard count / table metadata
`$C7A0-$C7DF / $C7F0`	ReadyShell RAM debug ring + head
`$C7E0-$C7E8`	global hotkey bindings

Why the layout matters

- Everything in `$1000-$C3FF` is replaceable.
- The shim survives all loads because it lives above the snapshot window.
- The hardware stack survives preload and enables the shim's `RTS`-based return path.
- Metadata in `$C600-$C9FF` stays visible without fetching a management bank from REU.

Resident Shim Byte Map

Page 1: $C800-$C8FF

- **`$C800-$C81F`** jumphelper entries including `$C810` log byte helper jump
- **`$C820-$C83F`** shim side ABI: target bank, filename, load end, current bank, last saved, bitmap bytes, storage drive, log index
- **`$C840-$C85F`** `load_disk`
- **`$C860-$C87F`** `load_reu`
- **`$C880-$C8BF`** `preload` sandwich
- **`$C8B0-$C8BF`** `stash_to_bank`
- **`$C8F0-$C8FF`** `fetch_bank`

Page 2: $C900-$C9FF

- **`$C900-$C93F`** `return_to_launcher`
- **`$C940-$C95F`** `switch_app`
- **`$C960-$C99F`** `debug_log_step` + padding
- **`$C9A0-$C9BF`** `reu_setup`
- **`$C9C0-$C9DF`** `set_bitmap`
- **`$C9E0-$C9FF`** `log_byte` ring writer

Jump / helper entry block

Address	Entry	Meaning
`$C800`	load disk + run	KERNAL load app and jump to `$1000`
`$C803`	load REU + run	fetch snapshot image and jump to `$1000`
`$C806`	run app	direct jump to `$1000`
`$C809`	preload	load app to REU and return to launcher caller
`$C80C`	return to launcher	stash current app and restore bank **0**
`$C80F`	switch app	stash current app and fetch target app
`$C812`	reserved helper slot	currently a placeholder jump slot in the resident shim, not a live stash helper
`$C815`	fetch helper	helper entry into REU fetch path
`$C818`	log byte helper	helper entry into shim debug ring

Shim data ABI

Address	Field	Active use
`$C820`	`target_bank`	preload / switch / fetch target bank
`$C821`	`filename_len`	launcher-written filename metadata
`$C822-$C823`	`app_size` scratch	written by launcher, not consumed by resident shim path today
`$C824-$C82F`	`filename[12]`	12 byte app token buffer for disk load paths
`$C830-$C831`	`load_end`	actual KERNAL `LOAD` end address captured by preload
`$C832-$C833`	unused	reserved padding
`$C834`	`current_bank`	read directly by apps and TUI helpers
`$C835`	`last_saved`	recovery hint for interrupted return path
`$C836-$C838`	bitmap tuplcolic	loaded state bits for banks **0-23**
`$C839`	`storage_drive`	shared file dialog default drive
`$C83A`	`log_index`	head byte for shim local ring at `$C880-$C99F`
`$C83B-$C83F`	reserved	unused padding
`$C840`	`load_disk` launcher-patched drive byte	
`$C89C`	device immediate	`preload` launcher-patched drive byte

ABI nuance that matters: launcher code still exposes `$C822-$C823` as a shim-side size field, but the resident shim path itself does not consume it today. The real preload result comes from `$C830-$C831`, where preload saves the KERNAL end address.

Annotated Resident Shim Walkthrough

This is the full functional resident shim walkthrough: jump block, data block, and every live routine. Padding bytes are omitted, but no active resident behavior is skipped.

```
; ------------------------------------------------------------------------
; Jump / helper block at $C800-$C81F
; ------------------------------------------------------------------------
$C800: JMP $C840   ; load_disk: disk LOAD then jump to $1000
$C803: JMP $C880   ; load_reu: fetch from target bank then jump to $1000
$C806: JMP $1000   ; run current image directly
$C809: JMP $C940   ; preload: stash launcher, load app, stash app, restore launcher
$C80C: JMP $C900   ; return_to_launcher
$C80F: JMP $C940   ; switch_app
$C812: JMP $C9A0   ; reserved jumpholder slot in current shim image
$C815: JMP $C8F0   ; fetch_bank helper
$C818: JMP $C960   ; log_byte helper

; ------------------------------------------------------------------------
; Data ABI at $C820-$C83F
; ------------------------------------------------------------------------
$C820: target_bank
$C821: filename_len
$C822-$C823: launcher scratch / app_size convenience (not consumed by resident shim)
$C824-$C82F: filename[12]
$C830-$C831: load_end (captured from KERNAL LOAD X/Y)
$C834: current_bank
$C835: last_saved
$C836-$C838: reu_bitmap lowhighvhi
$C839: storage_drive
$C83A: log_index
$C83B-$C83F: reserved

; ------------------------------------------------------------------------
; load_disk at $C840-$C87F
; ------------------------------------------------------------------------
$C840: LDA $C821           ; filename length
$C843: LDX #$24 / LDY #$C8 ; filename buffer at $C824
$C846: JSR $FFBD           ; SETNAM
$C848: LDA #$80            ; patched device immediate
$C84A: JSR $FFBA           ; SETLFS
$C84D: LDA #$01            ; patched device immediate
$C84F: LDY #$01
$C851: JSR $FFBA           ; SETLFS
$C854: LDA #$00            ; LOAD
$C856: JSR $FFD5           ; LOAD
$C859: JMP $1000

; ------------------------------------------------------------------------
; load_reu at $C860-$C87F
; ------------------------------------------------------------------------
$C860: LDA $C820           ; target bank
$C863: JSR $C8F0           ; fetch_bank
$C866: JMP $1000

; ------------------------------------------------------------------------
; preload at $C880-$C8BF
; ------------------------------------------------------------------------
$C880: LDA #$AA / STA $C807 ; debug marker: entered preload
$C885: LDA #$00 / JSR $C820 ; stash launcher image to bank 0
$C88A: LDA #$00 / STA $C880 ; launcher stashed
$C88F: LDA $C821            ; filename_len
$C892: LDX #$24 / LDY #$C8  ; filename buffer
$C895: JSR $FFBD            ; SETNAM
$C898: LDA #$00             ; patched device immediate
$C89B: LDX #$01             ; patched device immediate
$C89F: LDY #$FF             ; SETLFS
$C8A2: JSR $FFBA / STA $C809 ; about to LOAD
$C8A7: LDA #$00             ; LOAD into $1000
$C8A9: JSR $FFD5            ; LOAD into $1000
$C8AC: STX $C830 / STY $C831 ; save real KERNAL end address
$C8B2: LDA $C820 / JSR $C880 ; stash loaded app into target bank
$C8B5: LDA $C820 / JSR $C8F0 ; fetch bank
$C8B8: LDA $C820 / JSR $C9C0 ; set bitmap for target bank
$C8C3: LDA #$00 / JSR $C8F0  ; fetch launcher bank 0 back
$C8C6: LDA #$FF / STA $C80C  ; launcher restored
$C8C9: RTS

; ------------------------------------------------------------------------
; stash_to_bank at $C8B0-$C8BF
; fetch_bank   at $C8F0-$C8FF
; ------------------------------------------------------------------------
$C8B0: JSR $C9A0           ; reu_setup: bank in A, addr=$1000, len=$0600
$C8B3: LDA #$90 / STA $DF01 ; STASH command
$C8B8: RTS

$C8F0: JSR $C9A0
$C8F3: LDA #$91 / STA $DF01 ; FETCH command
$C8F8: RTS

; ------------------------------------------------------------------------
; return_to_launcher at $C900-$C93F
; ------------------------------------------------------------------------
$C900: LDA $C820 / JSR $C880 ; stash current app image
$C906: LDA $C834 / JSR $C9C0 ; set bitmap bit
$C90C: LDA $C835 / STA $C835 ; remember last_saved
$C912: LDA #$00 / JSR $C8F0  ; fetch launcher from bank 0
$C917: LDA #$00 / STA $C834  ; current=launcher
$C91C: JMP $1000

; ------------------------------------------------------------------------
; switch_app at $C940-$C95F
; ------------------------------------------------------------------------
$C940: LDA $C834 / JSR $C880 ; stash current app
$C946: LDA $C835 / STA $C835 ; last_saved=current
$C94C: LDA $C820 / JSR $C8F0 ; fetch target app
$C952: LDA $C820 / STA $C834 ; current=target
$C958: JMP $1000

; ------------------------------------------------------------------------
; debug_log_step at $C960-$C99F
; ------------------------------------------------------------------------
$C960: STA $D020           ; border color breadcrumb
$C963: STA $0F78           ; stack-page breadcrumb
$C966: STA $0428           ; top-left screen breadcrumb
$C969: CLC / ADC #$30      ; visible digit
$C96C: STA $0400           ; visible digit
$C96F: RTS

; ------------------------------------------------------------------------
; reu_setup at $C9A0-$C9BF
; ------------------------------------------------------------------------
$C9A0: STA $DF04           ; bank number
$C9A3: LDA #$00 / STA $DF02 ; C64 low byte = $00
$C9A8: LDA #$10 / STA $DF03 ; C64 high byte = $10 => $1000
$C9AD: STX $DF05           ; length low
$C9B4: LDA #$06 / STA $DF06 ; length high => $0600 bytes total
$C9B9: RTS

; ------------------------------------------------------------------------
; set_bitmap at $C9C0-$C9DF
; ------------------------------------------------------------------------
$C9C0: CMP #$18            ; only banks 0-23 tracked
$C9C2: BCS done
$C9C4: TAX
$C9C5: LDA #$07            ; bit index in byte
$C9C7: TAY
$C9C8: TXA
$C9C9: LSR / LSR / LSR     ; byte offset 0..2
$C9CC: TAX
$C9CD: LDA #$01            ; mask
$C9CF: DEY / BMI store_bit ; shift into target bit position
$C9D1: ASL A / JMP $C9C7
$C9D7: ORA $C836,X
$C9DA: STA $C836,X
$C9DD: RTS

; ------------------------------------------------------------------------
; log_byte at $C9E0-$C9FF
; ------------------------------------------------------------------------
$C9E0: PHA
$C9E1: LDX $00FC           ; preserve X
$C9E4: LDX $C83A           ; ring head
$C9E7: PLA
$C9E8: STA $C980,X         ; write into shim-local ring
$C9EB: INX / CPX #$20
$C9EE: BNE skip
$C9F1: LDX #$00            ; wrap at 32 bytes
$C9F3: STX $C83A           ; restore X
$C9F6: LDX $00FC
$C9F9: RTS
```

REU Ownership and Clipboard Accuracy

Current REU schema

Bank(s)	Role
0	launcher snapshot + launcher resume tail
0-23	bitmap-tracked app-slot namespace
24+	dynamic allocator pool
140-143	ReadyShell overlay/debug banks

Bank 0 split

Offset	Meaning
`$0000-$BDFF`	launcher snapshot image of C64 RAM `$1000-$C3FF`
`$0600-$FFFF`	launcher resume tail (`LauncherResumeV1` + header)

Truthful current wording: clipboard metadata is shared in C64 RAM, and clipboard payloads still use one dynamic REU bank per item. The single-bank packed clipboard design has not been implemented yet.

```
REU_APP_SNAPSHOT_SIZE = $0600
REU_RESUME_OFF        = $0600
REU_RESUME_FAIL_SIZE  = $0400
tail end              = $10000
```

Clipboard storage reality

- **Clipboard metadata in RAM:** `$C782` count, `$C718` table.
- **Per-item table layout:** each item stores REU bank id, type, and size.
- **Allocator behavior:** `clip_copy` calls `reu_alloc_bank(REU_CLIPBOARD)`, so payload banks come from the dynamic pool starting at 24.
- **Legacy symbolic contract:** some older headers/spec fields still name bank 1 as a clipboard bank, but active payload storage is not single-bank packed there.

apps.cfg and Launcher Lifecycle

Catalog reality

- Human-edited source: selected `cfg/profiles/<profile>.ini` catalog
- Generator/orchestrator: `build_support/readyos_profiles.py`
- Disk payload: `apps.cfg` on drive **8**
- Runtime consumer: launcher only
- Read mode: `apps.cfg:s,r`
- Parsed fields: drive, PRG token, display label, optional hotkey slot, description
- Assigned by launcher, not disk: REU bank identity

Boot never parses `apps.cfg`. Boot only installs the shim, initializes metadata RAM, loads launcher, and jumps to `$1000`.

Current release matrix: there are seven active profiles. `precog-dual-d71` is the default full-content verification target and the only one that uses `PREBOOT -> SETD71 -> BOOT`. The other six current profiles use direct `PREBOOT -> BOOT`.

Reload behavior

Cold / invalid-resume path

launcher starts → tries bank 0 resume payload → failure → parse `apps.cfg` from disk → validate → save fresh `LauncherResumeV1`

Normal return path

app returns via shim → bank 0 launcher image restored → launcher restarts → launcher restores cached state from bank 0 tail → `apps.cfg` usually not reread

Measured BSS and Headroom Snapshot

These values were rechecked from the current `obj/*.map` files in the April 8, 2026 tree. This is the evidence behind the "tight app" wording in the private markdown docs.

Tightest normal app images

App	Runtime end	BSS bytes	Headroom
`hixy`	`$0D38`	`6327`	`2759`
`tasklist`	`$4A3F`	`8944`	`7104`
`calcplus`	`$4637`	`1321`	`8136`
`calc20`	`$4048`	`4083`	`9655`
`simplecalc1s`	`$0EE8`	`2142`	`10015`

Linker-config nuance

- **Normal apps:** `cfg/ready_app.cfg` puts `BSS` directly inside the `$1000-$C3FF` app window.
- **Overlay app:** `readyshell` uses `cfg/ready_app_overlay.cfg`, not the normal app linker contract.
- **Current resident Readyshell image:** runtime end `$0874`, resident `BSS` = `171` bytes, headroom `10891` bytes to `$C600`.
- **Overlay guardrail:** release overlay size is `$2408`, debug overlay size is `$2408`, and the explicit C stack reserve prevents resident `BSS` from growing into the overlay execution window.

Transition Flows

Preload sandwich

1. Launcher writes target bank, filename, and drive into shim bytes.
2. Launcher calls `JSR $C809`.
3. Shim stashes launcher image to bank **0**.
4. Shim KERNAL loads target app into `$1000`.
5. Shim saves real load end to `$C830-$C831`.
6. Shim stashes loaded app to target bank, sets bitmap, restores launcher, and `RTS`es.

Return to launcher

1. App jumps to `$C80C` through TUI helper.
2. Shim stashes current app to its bank.
3. Shim updates bitmap and records `last_saved`.
4. Shim fetches launcher bank **0**.
5. Shim sets `current_bank` = **0** and jumps to `$1000`.
6. Launcher restarts and usually restores cached menu/catalog state from the bank 0 tail.

Launch from REU

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank **0**.
3. Launcher writes target scratch bytes and sets `current_bank`.
4. Launcher jumps to `$C803`.

Launch from disk

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank **0**.
3. Launcher writes filename and patched drive bytes.
4. Launcher sets `current_bank`.
5. Launcher jumps to `$C800`.

Warm Resume and App Nuances

Topic	Reality
Resume header	16-byte `RSM1` header with app id, schema, seq, and payload length
Launcher resume	`LauncherResumeV1` stores parsed catalog plus menu position in bank **0** tail
Snapshot vs resume	snapshot restores raw runnable image; resume tail restores validated high-level state
Tasklist	uses shared `resume_state` with segment-based save/load helpers

Debugging and Observability

Shim-local debug surfaces

Location	Meaning
`$C83A`	shim log head
`$C880-$C99F`	shim-local 32 byte debug ring
`$C960`	`debug_log_step` crash breadcrumb helper
`$0F78`	stack-page breadcrumb used by `debug_log_step`
`$0400`	screen breadcrumb used by `debug_log_step`

ReadyShell debug surfaces

Location	Meaning
`$C7A0-$C7DF`	ReadyShell RAM debug ring
`$C7F0`	ReadyShell RAM debug head
`0x4F0400`	ReadyShell REU debug head
`0x4F0F00`	ReadyShell REU debug data

The shim-local diagnostics exist for transition critical breadcrumbs in the always-on resident layer; the ReadyShell diagnostics exist for overlay/runtime analysis inside ReadyShell itself.

Source Priority and Drift Notes

1. `src/boot/boot_asm.s`
2. `src/apps/launcher/launcher.c`
3. `src/lib/resume_state.{h,c}`
4. `src/lib/reu_mgr*.{h,c}`
5. `src/lib/clipboard*.{c,h}`
6. `build_support/memory_map_spec.json`

- Older overlaying shim modules under `src/shim` are historical reference only.
- `src/shim/reu.h` still carries some legacy symbolic bank naming; docs must not mistake that for active clipboard payload behavior.
- Some older high-level banner comments near the top of `boot_asm.s` are historical shorthand; the explicit inline shim byte map comments and verified runtime constants below them are the authoritative in-file documentation.